Vhdl-parse

Documentation

Version 0.2.0, 2026-10-26

Sylvain Tertois

https://github.com/daixiwen/typst-vhdl-parse

Summary

`Vhdl-parse` is a typst module that parses VHDL files and extracts information from them that can be presented in a typst document. It makes writing documentation for VHDL code easier and lets use the VHDL code as single source of truth.

It only extracts information and provides it in structures. It is up to the user to format it in a way to present it in a readable manner.

Table of contents

1 Usage . . . 3
1.1 Invocation . . . 3
1.2 Comments . . . 3
1.3 Types . . . 3
2 Functions reference . . . 5
2.1 parse . . . 5
2.2 port-list . . . 6
2.3 generic-list . . . 7
2.4 fsm . . . 8
2.5 fsm-dot . . . 10
2.6 instances-list . . . 14
2.7 constants-list . . . 15
2.8 signals-list . . . 16
2.9 types-list . . . 17
3 Implementation details . . . 19
3.1 Architecture . . . 19
3.2 Caching . . . 19

4 History 20
4.1 Version 0.2.0 20
4.2 Version 0.1.0 20

1 Usage

1.1 Invocation

Import the `vhdl-parse` module as usual:

```typ
#import "@preview/vhdl-parse:0.2.0
```

Use the `parse()` function to parse your VHDL file.

```typ
#let parsed-file = vhdl-parse.parse(
    "test.vhd",
    read("test.vhd"))

#assert(parsed-file.messages.len() == 0, message. "errors were found in the VHDL file")
```

The object returned is used as parameter for all the other functions to extract different kind of information from the VHDL contents. The `message` field contains errors returned by the VHDL parser.

Please note that the `vhdl-parse` module does not present the extracted data in any ready-to-display state. It returns arrays and dictionary with the requested data, and it is up to the user to write a template to present the data in a pretty and consistent way across the documentation.

Please refer to section 2 for more details about all the functions that can be called with the parsed file.

1.2 Comments

When parsing VHDL, the module looks after comments to describe the objects it finds. It looks for both comments on the same line than the object, and for comments on the line before it, as shown in the following examples:

```vhdl
-- this is a leading comment (before the object)
signal my_signal: std_logic;

signal my_other_signal: std_logic; -- this is a trailing comment (with the object)
```

If an object has both a leading and a trailing comment, the parser doesn't know which one to pick. This is why there is a `comment-priority` parameter, which tells it which comment to pick when both are detected. The `parse()` function has the `comment-priority` parameter to give the default value, but it can then be overriden in all other function calls if needed.

1.3 Types

VHDL types are parsed and returned as 3 different objects to allow some flexibility in the presentation. For example the type `std_logic_vector(31 downto 0)` will be returned as:

- type name: `std_logic_vector`
- constraint or range: `(31 downto 0)`
- sizes: `("32",)`

Type name is the name of the type, excluding the constraint. Constraint is the constraint itself without the type name, and sizes is an array with one element per dimension, giving the size of the type in that dimension, if it is constrained.

The VHDL definition itself can always be reproduced by concatenating the type name and the constraint. The sizes array can be used to present the type size in a more readable way, avoiding the `to` or `downto` ranges.

For an array type such as

```vhdl
type my_type is array (0 to num_elements-1) of std_logic_vector(7 downto 0)
```

The returned elements for the array type are:

- subtype name: `std_logic_vector`
- range: `(0 to num_elements-1)(7 downto 0)`
- sizes: `("num_elements", "8")`

2 Functions reference

2.1 parse

Parses a VHDL file

The return value is a dictionary, used as parameter in the other functions to extract information from the parsed VHDL file. The `messages` element in the returned dictionary is a list of warnings or errors from the parser.

2.1.1 Example

```
#let parsed-file = vhdl-parse.parse(
    "test.vhd",
    read("test.vhd"))

Messages from the parser:

#if parsed-file.messages.len() == 0 [
   no messages
 ] else {
   for message in parsed-file.messages [
   - #message ]
 }
```

typ

```
Messages from the parser:

no messages
```

2.1.2 Parameters

```
parse(
  file-name: string,
  contents: string bytes,
  vhdl-variant: string int,
  comment-priority: string
) -> dictionary
```

file-name `string`

The VHDL file name

contents `string` or `bytes`

The VHDL code, usually the result of a `read` operation on the VHDL file

vhdl-variant `string` or `int`

(optional) The VHDL variant

Default: `2008`

comment-priority `string`

(optional) The default comment priority. Either “leading” or “trailing”, see section 1.2

Default: `"trailing"`

2.2 port-list

Returns the portlist from the first entity found in the parsed file

2.2.1 Return structure

an array of dictionaries, each item having the following elements:

name	type	description
`name`	`string`	the port name
`mode`	`string`	the port mode (in, out, inout, buffer)
`port-type`	`string`	the VHDL type of the port
`constraint`	`string or none`	the type constraint, see section 1.3
`sizes`	`array`	the type sizes, see section 1.3
`description`	`string or none`	a comment describing the port
`default-value`	`string or none`	the port default value

2.2.2 Example

```
#let ports = vhdl-parse.port-list(
  parsed-file)

Ports list:

#table(
   columns: (2cm, 1.5cm, 3.5cm, 1.5cm, 6cm),
   table.header([name], [mode], [type],
   [size], [description]),
   ..for entry in ports {
    ( [#entry.name],
      [#entry.mode],
      [#entry.port-type],
      [#entry.sizes.join("x")],
      [#entry.description])
    }
)
```

Ports list:

name	mode	type	size	description
clock	in	std_logic		main clock
sreset	in	std_logic		main reset, synchronous, active high
output_a	out	std_logic		a regular output
input_b	in	std_logic		one input
output_c	out	std_logic		another output
input_d	in	std_logic		another input
data_in	in	std_logic_vector	16	
data_out	out	std_logic_vector	16	data out

2.2.3 Parameters

```
port-list(
  parsed-file: dictionary,
  comment-priority: string none
) -> array
```

parsed-file `dictionary`

The parsed file object, as returned by `parse()`

comment-priority `string` or `none`

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: `none`

2.3 generic-list

Returns the generics list from the first entity found in the parsed file

2.3.1 Return structure

an array of dictionaries, each item having the following elements:

name	type	description
`name`	`string`	the generic name
`generic-type`	`string`	the VHDL type of the generic
`constraint`	`string or none`	the type constraint, see section 1.3
`sizes`	`array`	the type sizes, see section 1.3
`description`	`string or none`	a comment describing the generic
`default-value`	`string or none`	the generic default value

2.3.2 Example

```
#let generics = vhdl-parse.generic-list(   typ
  parsed-file)

Generics list:

#table(
    columns: (2cm, 3.5cm, 1.5cm, 6cm, 2cm),
    table.header([name], [type], [size],
    [description], [default]),
    ..for entry in generics {
     ( [#entry.name],
       [#entry.generic-type],
       [#entry.sizes.join("x")],
       [#entry.description],
       [#entry.default-value])
     }
)
```

Generics list:

name	type	size	description	default
flag	boolean		a flag, can be true or false	
value	std_logic_vector	16	a value with a default	x"DEAD"

2.3.3 Parameters

```
generic-list(
  parsed-file: dictionary,
  comment-priority: string none
) -> array
```

parsed-file `dictionary`

The parsed file object, as returned by `parse()`

comment-priority `string` or `none`

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: `none`

2.4 fsm

Returns information about a state machine found in the parsed file

This function looks for a case statement and assignments within the different cases:

```vhdl
case my_state is
  when state_1 =>
    next_state <= state_2

  when state_2 =>
    next_state <= state_1

  when others =>
    next_state <= state_1
end case;
```

Both signal and variable assignments are detected. The signal or variable that is holding the current state (`my_state` in the example above) is defined with the `read-variable-name` parameter in the function call, while the signal or variable assigned to the next state (`next_state` in the example above) is defined with the `write-variable-name` parameter. If the same signal is both read and written, the name just needs to be defined in `read-variable-name` and `write-variable-name` can be kept to `none` (the default). If an assignment is found in the `when others` clause, the destination of the assignment will be stored as the default state.

`fsm()` will attempt to find a description for each transition. For that it will look for a comment near the **condition** (an `if`, `elsif` or `else` statement) before the assignment. If it doesn’t find a comment, it will attempt to use the test condition itself (inside the `if` or `elsif`) as description.

2.4.1 Return structure

a dictionary with the following elements:

name	type	description
`default-state`	`string`	the default state (i.e. found in an "others" case)
`states`	`array`	an array of states

The `states` array contains dictionaries, with each item having the following elements:

name	type	description
`name`	`string`	the state name
`description`	`string or none`	the comment describing the state
`transitions`	`array`	an array of transitions

The `transitions` arrays contain dictionaries, with each item having the following elements:

name	type	description
`destination`	`string`	the state it is transitioned to
`condition`	`string`	the condition for the transition, if found or an empty string
`description`	`string or none`	the comment describing the condition, if found

2.4.2 Example

```
#let fsm = vhdl-parse.fsm(parsed-file,"fsm_state")
typ

List of states:

#table(
    columns: (3cm, 7cm),
    table.header([state], [description]),
    ..for state in fsm.states {
        ( [#state.name], [#state.description])
    }
)
```

List of states:

state	description
reset	start and reset state
idle	normal state when nothing happens
read_input	waiting for input
write_output	send output

2.4.3 Parameters

```
fsm(
  parsed-file: dictionary,
  read-variable-name: string,
  write-variable-name: string none,
  comment-priority: string none
) -> dictionary
```

parsed-file `dictionary`

The parsed file object, as returned by `parse()`

read-variable-name `string`

The name of the signal or variable holding the current fsm state

write-variable-name `string` or `none`

(optional) The name of the signal or variable holding the next fsm state, if different from read-variable-name

Default: `none`

comment-priority `string` or `none`

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: `none`

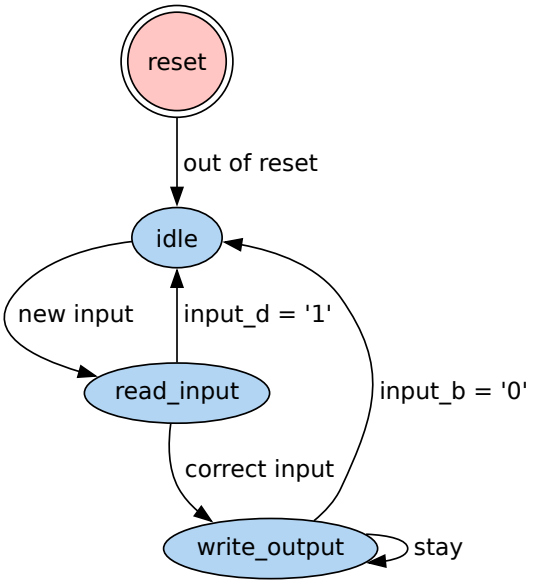
2.5 fsm-dot

Returns a description of an FSM in the DOT format, ready to be drawn by the diagraph package.

For more information about FSM detection please refer to the `fsm()` function.

2.5.1 Example

```
#let dot = vhdl-parse.fsm-dot(
  parsed-file,
  "fsm_state",
  font-name: "DejaVu Sans,
Helvetica,Arial,sans-serif",
  state-background-color: blue.lighten(70%),
  default-state-background-color:
red.lighten(70%)
  )
#import "@preview/diagraph:0.3.7"
#diagraph.render(dot)
```



2.5.2 Parameters

```
fsm-dot(
  parsed-file: dictionary,
  read-variable-name: string,
  write-variable-name: string none,
  comment-priority: string none,
  left-to-right: bool,
  font-name: string,
  state-shape: string,
  state-background-color: color,
  state-line-color: color,
  state-text-color: color,
  state-font-size: float,
  default-state-shape: string,
  default-state-background-color: color none,
  default-state-line-color: color none,
  default-state-text-color: color none,
  default-state-font-size: float none,
  transition-line-color: color none,
  transition-text-color: color none,
  transition-font-size: float none
) -> string
```

parsed-file `dictionary`

The parsed filed object, as returned by `parse()`

read-variable-name `string`

The name of the signal or variable holding the current fsm state

write-variable-name `string` or `none`

(optional) The name of the signal or variable holding the next fsm state, if different from `read-variable-name`

Default: `none`

comment-priority `string` or `none`

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: `none`

left-to-right `bool`

(optional) if true, make a left-to-right diagram instead of top-to-down

Default: `false`

font-name `string`

(optional) the font to use (has to be accessible to Typst)

Default: `"Helvetica,Arial,sans-serif"`

state-shape `string`

(optional) the shape to use for states (refer to the Graphviz documentation for a list)

Default: `"ellipse"`

state-background-color `color`

(optional) the background color for states

Default: `white`

state-line-color `color`

(optional) the line color for states

Default: `black`

state-text-color `color`

(optional) the color of text for states

Default: `black`

state-font-size `float`

(optional) the text size for states (in points)

Default: `14`

default-state-shape `string`

(optional) the shape to use for the default state (refer to the Graphviz documentation for a list)

Default: `"doublecircle"`

default-state-background-color `color` or `none`

(optional) the background color for the default state. If none, use the same one as for the regular states

Default: none

default-state-line-color `color` or `none`

(optional) the line color for the default state. If none, use the same one as for the regular states

Default: none

default-state-text-color `color` or `none`

(optional) the color of text for the default state. If none, use the same one as for the regular states

Default: none

default-state-font-size `float` or `none`

(optional) the text size for the default state (in points). If none, use the same one as for the regular states

Default: none

transition-line-color `color` or `none`

(optional) the line color for the transitions. If none, use the same one as for the regular states

Default: none

transition-text-color `color` or `none`

(optional) the color of text for the transitions. If none, use the same one as for the regular states

Default: none

transition-font-size `float` or `none`

(optional) the text size for the transitions (in points). If none, use the same one as for the regular states

Default: none

2.6 instances-list

Returns the instances list from the first entity found in the parsed file

2.6.1 Return structure

an array of dictionaries, each item having the following elements:

name	type	description
`label`	`string`	the instantiation label
`entity`	`string`	the entity being instantiated
`description`	`string or none`	a comment describing the instance
`generics-map`	`array`	an array of generics
`ports-map`	`array`	an array of ports

`generics-map` and `ports-map` are arrays of dictionaries, each item having the following elements:

name	type	description
`origin`	`string`	name of the generic or port on the entity side
`expression`	`string`	the expression assigned to the generic or port

2.6.2 Example

```
1  List of Instances                                    typ
2
3  #let instances = vhdl-parse.instances-
   list(parsed-file)
4
5  #table(
6      columns: (3cm, 3cm, 7cm),
7      table.header([label], [entity],
   [description]),
8      ..for element in instances {
9          ( [#element.label], [#element.entity],
           [#element.description])
10     }
11 )
```

List of Instances

label	entity	description
my_instance	work.comp	an example instantiation

2.6.3 Parameters

```
instances-list(
  parsed-file: dictionary,
  comment-priority: string none
) -> array
```

parsed-file `dictionary`

The parsed file object, as returned by `parse()`

comment-priority `string` or `none`

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: `none`

2.7 constants-list

Returns the constants list from the architecture or package definition in the parsed file

2.7.1 Return structure

an array of dictionaries, each item having the following elements:

name	type	description
`name`	`string`	the constant name
`object-type`	`string`	the constant type
`constraint`	`string or none`	the type constraint, see section 1.3
`sizes`	`array`	the type sizes, see section 1.3
`expression`	`string or none`	the constant value
`description`	`string or none`	a comment describing the constant

2.7.2 Example

```
1  List of Constants                                typ
2
3  #let constants = vhdl-parse.constants-
   list(parsed-file)
4
5  #table(
6      columns: (2cm, 3.5cm, 1.5cm, 3cm, 4cm),
7      table.header([name], [type], [size],
   [description], [value]),
8      ..for entry in constants {
9          ( [#entry.name],
10           [#entry.object-type],
11           [#entry.sizes.join("x")],
12           [#entry.description],
13           [#entry.expression])
14     }
15 )
```

List of Constants

name	type	size	description	value
c_ones	std_logic_vector	32	a constant	(others => '1')
num_elements	natural			16
num_elements	natural			8

2.7.3 Parameters

```
constants-list(
  parsed-file: dictionary,
  comment-priority: string none
) -> array
```

parsed-file `dictionary`

The parsed file object, as returned by `parse()`

comment-priority `string` or `none`

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: `none`

2.8 signals-list

Returns the signals list from the architecture or package declaration in the parsed file

2.8.1 Return structure

an array of dictionaries, each item having the following elements:

name	type	description
`name`	`string`	the signal name
`object-type`	`string`	the signal type
`expression`	`string or none`	the signal initial value
`constraint`	`string or none`	the type constraint, see section 1.3
`sizes`	`array`	the type sizes, see section 1.3
`description`	`string or none`	a comment describing the signal

2.8.2 Example

```
List of Signals                                    typ

#let signals = vhdl-parse.signals-list(parsed-file)

#table(
    columns: (2cm, 3.5cm, 1.5cm, 5cm, 3.5cm),
    table.header([name], [type], [size], [description], [value]),
    ..for entry in signals {
        ( [#entry.name],
          [#entry.object-type],
          [#entry.sizes.join("x")],
          [#entry.description],
          [#entry.expression])
    }
)
```

List of Signals

name	type	size	description	value
fsm	fsm_wrapper_t		state machine signal	
mysignal	unsigned	16	a signal with a comment on the same line	(others => '0')

2.8.3 Parameters

```
signals-list(
  parsed-file: dictionary,
  comment-priority: string none
) -> array
```

parsed-file `dictionary`

The parsed file object, as returned by `parse()`

comment-priority `string` or `none`

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: `none`

2.9 types-list

Returns the types list from the architecture or package declaration in the parsed file

2.9.1 Return structure

an array of dictionaries, each item having the following elements:

name	type	description
`name`	`string`	the type name
`kind`	`string`	the type kind: "enumeration", "record", "subtype" or "array"
`description`	`string or none`	a comment describing the type
`definition`	`see below`	an object with the type definition. The structure depends on the type kind

The definition object is different depending on the type kind:

- Enumeration: an array of dictionaries, each item having the following elements:

name	type	description
`element-name`	`string`	the enumeration element name
`description`	`string or none`	a comment describing the element

- Record: an array of dictionaries, each item having the following elements:

name	type	description
`element-name`	`string`	the record element name
`element-type`	`string`	the record element type

name	type	description
`description`	`string or none`	a comment describing the record element

- Subtype: a dictionary with the following elements:

name	type	description
`subtype`	`string`	the type name
`constraint`	`string or none`	the subtype constraint, see section 1.3
`sizes`	`array`	the subtype sizes, see section 1.3

- Array: a dictionary with the following elements:

name	type	description
`range`	`string or none`	the array ranges as VHDL, see section 1.3
`sizes`	`array`	the array sizes, see section 1.3
`subtype`	`string`	the array element type

2.9.2 Example

```
1  List of types                                                typ
2
3  #let types = vhdl-parse.types-list(parsed-
   file)
4
5  #table(
6      columns: (3.5cm, 3cm, 6cm),
7      table.header([name], [kind],
   [description]),
8      ..for entry in types {
9          ( [#entry.name], [#entry.kind],
           [#entry.description] )
10     }
11 )
```

List of types

name	kind	description
fsm_t	enumeration	enumeration type for the FSM
fsm_wrapper_t	record	a wrapper around the fsm type to test record element access in FSM detection
byte_t	subtype	a subtype
matrix_t	array	an array

2.9.3 Parameters

```
types-list(
  parsed-file: dictionary,
  comment-priority: string none
) -> array
```

parsed-file `dictionary`

The parsed file object, as returned by `parse()`

comment-priority `string` or `none`

(optional) override the default comment priority, either “leading” or “trailing”, see section 1.2

Default: `none`

3 Implementation details

This section is only relevant if you are interested in the inner workings of the package or if you want to make any changes.

3.1 Architecture

The package is made of a lightweight Typst file calling a WASM plugin. the plugin itself is written in Rust and uses the `vhdl_lang` crate. The plugin and the Typst file communicate parameters and return values using byte arrays, and the CBOR protocol is used on both sides to transfer more complex structures.

There is a very basic `AGENTS.md` file in the repository. I used OpenCode to kickstart the project because I couldn't find how to use the `vhdl_lang` crate by myself, and to write some tests. The rest was written manually, because it is pretty basic code, walking through the structures returned by `vhdl_lang` to find the relevant information and putting it in structures which can be serialized for Typst. I found it easier to just write the code than to try to det en LLM to put it exactly in the way I wanted to. Maybe I'm just bad at prompting.

3.2 Caching

The plugin caches results from a VHDL parsing, because I thought it would be very inefficient to redo it at each function call. Unfortunately it makes it difficult to make the plugin pure as required by Typst: a plugin function call must not have any observable side effects on future plugin calls, and given the same arguments, it must always return the same value.

I have therefore implemented a cache system. The `parse()` function stores the results of the parsing in a static cache, using the lazystatic. It returns an ID for typst, and the other functions use that ID as a parameter, so that the parsed VHDL can be extracted again from the cache. To respect the purity requirement, the other functions also send the VHDL file name and the contents with the ID. That way the plugin can parse the VHDL file again if needed, and technically the function is not dependent on previous calls, but still benefits from a cache.

4 History

4.1 Version 0.2.0

4.1.1 Fixes, breaking changes

The `types-list()` function returned dictionaries with element names using snake_case (such as `element_type`) instead of camel-case (such as `element-type`), contrary to what was indicated in the documentation. This is now fixed, but any script relying on the snake_case items from version 0.1.0 need to be changed.

4.1.2 Improvements

For VHDL types, added a `sizes` array which present the type size in a more readable way than a raw VHDL range. See section 1.3 for details.

4.2 Version 0.1.0

First Release